

Design Pattern Recognition Exam

Syllabus: Adapter • Decorator • Composite • Mediator • State • Strategy

August 4, 2026

Instructions

For each problem below, identify which design pattern(s) from the syllabus — Adapter, Decorator, Composite, Mediator, State, or Strategy — would be the appropriate solution. **Some problems require more than one pattern working together**; state all that apply. Justify your answer in one to two sentences: name the specific detail in the problem statement that points to that pattern, not just the pattern's name. Problems are arranged roughly easy to hard. An answer key with justifications follows at the end — attempt every problem before consulting it.

Time suggested: 90 minutes. **Total problems:** 20.

1 Problems

1. A mobile app lets users choose how their route is calculated: shortest distance, fastest time, or fewest turns. The choice is made once from a settings menu before the route is computed, and each calculation method is a self-contained, independent algorithm. *[Easy]*
2. An e-commerce site sells build-your-own sandwiches. A base sandwich has a price and description; customers can add extra cheese, bacon, avocado, or a double portion of any topping, in any combination, and the receipt must itemize and total the final price without a separate class existing for every possible combination. *[Easy]*
3. A company's org chart needs a single `totalHeadcount()` call at the CEO node to return the correct number of employees anywhere beneath them, no matter how many management layers deep the tree goes, and the same call must also work correctly if pointed at a single individual contributor with no reports. *[Easy]*
4. A ride-hailing app's driver and rider apps never exchange messages directly with each other's devices. Instead, both report to a central dispatch server, which decides who gets matched with whom and pushes updates to both sides. *[Easy]*
5. A traffic-light-controlled pedestrian crossing button does nothing while the light is already Red-for-cars, immediately triggers a countdown if the light is Green-for-cars and no request is pending, and is ignored if a request is already pending. The light itself tracks which of these situations currently applies. *[Easy]*
6. Your analytics dashboard needs to pull data from an older internal reporting tool that only exposes `fetchStats(String reportId)` returning raw CSV text, but every other data source in your dashboard is built around a `DataSource` interface with a `getMetrics(): List<Metric>` method. You are not permitted to modify the older reporting tool. *[Easy]*
7. A word processor's "spell check" feature can run one of three algorithms depending on the user's selected language dictionary (English, Spanish, French), chosen from a menu, with no algorithm aware of or dependent on the others. *[Easy-Medium]*
8. A video-conferencing app's mute button, camera-toggle button, and "raise hand" button must never hold direct references to each other's UI state, but pressing mute while the camera is already off should also visually gray out the "background blur" option — and this cross-widget rule needs to live in exactly one place so new buttons can be added later without rewiring existing ones. *[Medium]*
9. A logging framework lets a base log writer be wrapped with any combination of: a timestamp-prefixer, a JSON-formatter, and a rate-limiter, applied in an order the application configures

at startup, where getting the order wrong (e.g. JSON-formatting after rate-limiting vs. before) visibly changes the output. *[Medium]*

10. An online order's allowed actions change completely depending on whether it is Placed, Shipped, Delivered, or Cancelled — for instance, "cancel" is only valid while Placed, and "request refund" is only valid once Delivered — and these rules are checked in several different methods (`cancel()`, `refund()`, `ship()`) that would otherwise all need the same repeated conditional. *[Medium]*
11. A drawing app represents shapes as a tree (groups can contain shapes and other groups, recursively, for a single `getBoundingBox()` call to work at any level) and additionally lets the user attach an optional drop-shadow effect to any individual shape *or* to an entire group, independently of the tree structure. *[Medium]*
12. A payment module must support Stripe, PayPal, and a legacy in-house billing system, each with a completely different method signature for actually charging a card, unified behind one `PaymentGateway` interface the rest of the checkout flow calls without knowing which underlying provider is active. *[Medium]*
13. A music-streaming app's playback controls (play/pause, skip, seek bar) behave differently depending on whether the app is Buffering, Playing, Paused, or Offline — and separately, none of the three controls should ever need to check each other's enabled/disabled status directly; a single `PlayerController` object is the only thing that inspects current playback status and updates all three controls' appearance. *[Medium-Hard]*
14. A file-compression utility lets users chain any combination of encryption and compression around a base file stream, where the two effects must be layered in a specific order to work correctly, and separately needs to read from a third-party cloud-storage SDK whose file-read method (`download(byte[] buf, long off, int len)`) doesn't match the plain `InputStream`-like interface the rest of the compression pipeline expects to wrap. *[Hard]*
15. A smart-home hub coordinates lights, thermostat, and door locks so that no single device ever directly commands another (all coordination flows through the hub), and every device's actual behavior in response to a hub command depends on whether the whole system is currently in Home, Away, or Night mode — e.g., motion sensors trigger lights only in Home mode. *[Hard]*
16. A game character's enemy AI has three health-based combat modes (Healthy, Wounded, Enraged) that change its available actions over time as it takes damage, and separately, within whichever mode it's in, the specific attack it executes each turn is chosen by a swappable difficulty-dependent algorithm (aggressive, defensive, or randomized) that the game designer can plug in per enemy type without touching the health-mode logic. *[Hard]*
17. A pricing engine for a subscription box service: bundles can contain individual products or

nested sub-bundles (recursively, for one `getTotalPrice()` call to work at any nesting level), each individual product can have an optional gift-wrap or expedited-processing surcharge layered on independently, and the discount actually applied to a computed total is chosen at checkout time from several interchangeable, swappable discount algorithms (seasonal, loyalty-tier, first-time-customer) picked based on the logged-in customer. *[Hard]*

18. A UI form-builder library needs its checkbox/dropdown/slider widgets to coordinate cross-widget enable/disable rules through one central form controller (no widget references another widget directly), and separately, some of those same widgets are actually third-party components from a widget library whose event-callback signatures (`onValueChanged(Object src, Object oldVal, Object newVal)`) don't match the simple `onChange(Widget)` interface the form controller expects every widget to expose. *[Hard]*
19. A document object has Draft, InReview, and Published stages that determine what `edit()` and `publish()` do and which stage comes next, and separately, exporting that same document can produce plain text, Markdown, or PDF output depending on a format chosen by the user at export time, completely independent of what stage the document is currently in. *[Medium-Hard]*
20. A logistics platform: shipments are organized as containers that may hold individual packages or nested sub-containers, with one `totalWeight()` call needing to work at any level; each package can independently have "fragile" and "temperature-controlled" handling layered onto its base cost, combinable in any order; the platform must integrate with three regional carriers whose booking APIs have incompatible method signatures, unified behind one internal interface; and which carrier is actually booked for a given shipment is decided by a runtime-swappable optimization algorithm (cheapest, fastest, or lowest-emissions) that the client selects. *[Very Hard]*
21. A multiplayer game server: each connected player's client never messages another player's client directly — all game actions are routed through a central game-session object that validates and broadcasts them. Each player object has a lifecycle (Connecting, Lobby, InMatch, Disconnected) that determines which actions are currently valid for them, checked identically across several different action-handling methods. The server also supports player inventories that can contain individual items or nested "container" items (like a backpack holding other items) with one `getInventoryWeight()` call needing to work regardless of nesting. Finally, matchmaking picks opponents using one of several swappable ranking algorithms (skill-based, latency-based, or pure-random) configurable per game mode. *[Very Hard]*

2 Answer Key

1. **Strategy.** Three independent, interchangeable route-calculation algorithms, chosen once from outside (the settings menu) with no notion of transitioning between them — the algorithms don't know about each other at all.
2. **Decorator.** Freely combinable, repeatable (“double portion”) optional add-ons layered around a stable base object without a class per combination — textbook subclass-explosion avoidance.
3. **Composite.** “The same call must also work correctly if pointed at a single individual contributor” is precisely the leaf-and-tree-treated-uniformly guarantee that defines Composite.
4. **Mediator.** Riders and drivers never talk directly; a central dispatch server decides matching and routes all communication — classic hub-and-spoke.
5. **State.** A small set of well-defined situations (no request pending / request pending / already red) with behavior on the same button-press event differing entirely based on which one currently applies, and the light itself tracks and transitions between them.
6. **Adapter.** Two fixed, incompatible interfaces (raw-CSV-returning `fetchStats` vs. the app-wide `DataSource.getMetrics()`) that need pure translation, with the legacy tool off-limits to modify — no new behavior is being added, just a shape conversion.
7. **Strategy.** Same shape as problem 1: interchangeable, independent algorithms selected externally (dictionary choice) with no shared state or transition logic between them.
8. **Mediator.** The explicit “must never hold direct references to each other's state” plus “this cross-widget rule needs to live in exactly one place” is the textbook GUI-form Mediator setup.
9. **Decorator.** Freely-orderable, combinable wrapper layers around a base writer, where the problem explicitly flags that order changes the observable output — exactly the ordering caveat that comes with stacking decorators.
10. **State.** The same repeated conditional logic across multiple methods (`cancel()`, `refund()`, `ship()`), all keyed on the same underlying stage variable, is precisely the scattered-conditional problem State is built to eliminate by moving each stage into its own class.
11. **Composite + Decorator.** The recursive group/shape tree with a uniform `getBoundingBox()` call is Composite. The drop-shadow effect attachable to *either* an individual shape *or* an entire group, independent of the tree structure, is Decorator layered on top of whatever **Component** (leaf or composite) it's given.
12. **Adapter.** Three payment providers with incompatible method signatures unified behind one

interface the rest of the app already depends on — one adapter per provider, pure translation, no added behavior.

13. **State + Mediator.** Playback-status-dependent control behavior (Buffering/Playing/Paused/Offline) is State. The requirement that the three controls never check each other directly, with a single `PlayerController` as the sole arbiter of shared appearance updates, is Mediator layered on top — the controller’s own decisions are driven by whichever playback State it currently holds.
14. **Decorator + Adapter.** The combinable, order-sensitive encryption/compression chain is Decorator. The mismatch between the third-party cloud SDK’s `download(buf, off, len)` signature and the plain stream interface the decorators expect to wrap is a separate translation problem, requiring an Adapter at the bottom of the decorator chain before any layers can be stacked on top of it.
15. **Mediator + State.** “No single device ever directly commands another...all coordination flows through the hub” is Mediator. “Every device’s actual behavior...depends on whether the whole system is currently in Home, Away, or Night mode” is State — here the interesting twist is that the *mediator itself* is the one holding/consulting the current System State to decide how to interpret and route each command, so the two patterns are stacked rather than independent.
16. **State + Strategy.** The three health-based modes that change over time as damage accumulates (Healthy/Wounded/Enraged) is State. “Within whichever mode it’s in, the specific attack...is chosen by a swappable...algorithm...plugged in per enemy type without touching the health-mode logic” is Strategy, deliberately kept independent of the state logic — each State object would internally delegate to whatever attack Strategy the enemy was configured with.
17. **Composite + Decorator + Strategy.** Nested bundle/sub-bundle/product tree with one `getTotalPrice()` call = Composite. Independently-layerable gift-wrap/expedited surcharges on individual products = Decorator. The discount algorithm chosen at checkout time from several interchangeable options based on customer type = Strategy, applied on top of whatever total the Composite/Decorator combination produced.
18. **Mediator + Adapter.** Central form controller arbitrating cross-widget rules with no widget referencing another directly = Mediator. Third-party widgets whose `onValueChanged(src, oldVal, newVal)` callback shape doesn’t match the controller’s expected `onChange(Widget)` interface = Adapter, needed so those third-party widgets can participate as ordinary colleagues in the mediator relationship at all.
19. **State + Strategy.** Document lifecycle stages (Draft/InReview/Published) governing `edit()/publish()` and their own transitions is State. The export-format choice (text/Markdown/PDF), made externally and completely orthogonal to what stage the document is in, is Strategy — a useful contrast case showing the two patterns can coexist on the same object addressing entirely unrelated concerns.

20. **Composite + Decorator + Adapter + Strategy.** Container/package weight tree = Composite. Freely-combinable fragile/temperature-control cost add-ons = Decorator. Three carriers with incompatible booking signatures unified behind one interface = Adapter (one per carrier). The runtime-swappable carrier-optimization algorithm (cheapest/fastest/lowest-emissions) that decides *which* adapter to invoke = Strategy layered on top. All four concerns are independent of one another and none substitutes for the others.
21. **Mediator + State + Composite + Strategy.** Players never messaging each other directly, routed through a central game-session object = Mediator. Per-player lifecycle (Connecting/Lobby/InMatch/Disconnected) governing which actions are valid, checked identically across multiple methods = State. Nestable inventory items (backpacks holding items) with one uniform `getInventoryWeight()` call = Composite. Swappable matchmaking ranking algorithms configurable per game mode = Strategy. This is the syllabus's full four-pattern ceiling case: each concern is genuinely independent and would be designed and tested separately, then composed.